

Tag 6

Git und GitHub

Versionen speichern und das Projekt teilen. Teil 1: Git einrichten, init, add, commit, log, diff und restore. Teil 2: alte Stände, switch, Branches und merge. Teil 3: GitHub-Konto, SSH-Schlüssel, push, jemanden einladen, pull und clone.

Warum das wichtig ist

An Tag 4 hast du das Skript `backup.sh` geschrieben. Es läuft. Jetzt willst du etwas Neues einbauen. Aber was ist, wenn das Skript danach nicht mehr läuft?

Viele machen dann zuerst eine Kopie:

```
cp backup.sh backup_alt.sh
```

Dann ändern sie `backup.sh`. Am nächsten Tag kommt die nächste Änderung. Und wieder eine Kopie. Nach einer Woche sieht der Ordner so aus:

```
ls
# backup_alt.sh  backup_final2.sh  backup_final.sh  backup_neu.sh  backup.sh
```

Welche Datei ist die richtige? Was ist der Unterschied zwischen `backup_final.sh` und `backup_final2.sh`? Welche lief zuletzt ohne Fehler? Das weiß nach einer Woche niemand mehr.

Das gibt es nicht nur bei Skripten. Vielleicht kennst du es von Dokumenten: `Bewerbung.docx`, `Bewerbung_neu.docx`, `Bewerbung_final.docx`. Das ist dasselbe Problem.

Git löst das. Du hast nur noch eine Datei: `backup.sh`. Ist ein Stand gut, sagst du Git: Merke dir diesen Stand. Git speichert ihn mit Datum, mit deinem Namen und mit einem kurzen Satz, was du geändert hast. Geht später etwas kaputt, holst du einen alten Stand zurück. Und du siehst genau, welche Zeile sich geändert hat.

Teil 1: Git auf deinem Rechner bis zur ersten Pause

Was ist Git?

Git ist ein Programm. Es speichert Versionen deiner Dateien. Eine gespeicherte Version heißt **Commit**. Ein Commit merkt sich, was sich geändert hat, wer es war und wann. Dazu schreibst du eine kurze Nachricht, was du gemacht hast.

Linus Torvalds hat Git 2005 für den Linux-Kernel geschrieben. Seit Juli 2005 pflegt Junio Hamano das Programm. Git ist freie Software. In einer großen Umfrage von 2022 sagten 93 Prozent der Entwickler: Ich nutze Git.

Git installieren und einrichten

```
sudo apt update
sudo apt install git
git --version
# git version 2.43.0
```

Hinweis: `sudo` und `apt` lernst du an Tag 8 und 9. Tippe die zwei Zeilen jetzt einfach ab. `sudo` fragt nach deinem Linux-Passwort. Beim Tippen siehst du nichts. Fragt `apt` `Do you want to continue? [Y/n]`, drücke Enter.

Dann sagst du Git drei Dinge. Das machst du nur einmal pro Rechner:

```
git config --global user.name "Carlos Ruiz"
git config --global user.email "carlos@example.com"
git config --global init.defaultBranch main
```

Name und E-Mail schreibt Git in jeden Commit. Nimm die E-Mail, mit der du dich bei GitHub anmeldest. Die dritte Zeile gibt der Reihe deiner Commits den Namen `main`. Diese Reihe heißt **Branch**. GitHub nutzt denselben Namen. `--global` heißt: gilt für alle deine Repos. Prüfen kannst du alles mit `git config --list`.

Ein Ordner wird ein Repo

Ein **Repository**, kurz **Repo**, ist ein Ordner, dessen Versionen Git speichert. Dein Kursordner `~/linux-kurs` wird heute ein Repo.

```
cd ~/linux-kurs
git init
# Initialized empty Git repository in /home/carlos/linux-kurs/.git/
```

`git init` legt den versteckten Ordner `.git` an. Du siehst ihn mit `ls -a`. Darin liegen alle Versionen. Fass ihn nicht an. Löschst du `.git`, ist der ganze Verlauf weg.

Was Git gerade sieht, zeigt `git status`:

```
git status
# ...
# Untracked files:
#   tag02/
#   tag03/
#   tag04/
#   tag05/
```

"Untracked" heißt: Diese Dateien speichert Git noch nicht.

Eine Version speichern

Das geht in zwei Schritten:

1. `git add` wählt aus, was in den nächsten Commit kommt.
2. `git commit -m "Nachricht"` speichert es als Version.

```
git add .
git commit -m "Add work from week 1"
# [main (root-commit) d1fde11] Add work from week 1
# 3 files changed, 5 insertions(+)
```

Der Punkt in `git add .` heißt: alles in diesem Ordner. Mit `git add tag05/woche1.sh` wählst du nur eine Datei. So bestimmst du, was zusammen in einen Commit gehört. Git nennt diesen Zwischenschritt **Staging**.

Die Nachricht hinter `-m` sagt, was der Commit macht. Sie steht im Imperativ, also als Befehl: `Add backup script`, nicht `Added backup script`. So will es auch die Klausur.

Verlauf und Änderungen ansehen

`git log --oneline` zeigt alle Commits, der neueste steht oben:

```
git log --oneline
# 09848b4 (HEAD -> main) Add goodbye message
# adfc1ed Add hello script
# d1fde11 Add work from week 1
```

Vorne steht die Nummer des Commits. `HEAD -> main` heißt: Hier bist du gerade. Ohne `--oneline` zeigt `git log` auch, wer und wann.

`git diff` zeigt, was du seit dem letzten Commit geändert hast:

```
git diff
# diff --git a/tag06/hallo.sh b/tag06/hallo.sh
# ...
# echo "Hallo $USER"
# +echo "Bis morgen"
```

Wichtig sind die Zeilen unten. `+` heißt: Zeile neu. `-` heißt: Zeile gelöscht. Nach `git add` zeigt `git diff` diese Änderung nicht mehr.

Eine Änderung zurücknehmen

Du hast aus Versehen `>` statt `>>` getippt. Dein Skript ist überschrieben. Git holt die Datei zurück, so wie sie im letzten Commit war:

```
git restore hallo.sh
```

Alles, was du seit dem letzten Commit an der Datei geändert hast, ist dann weg. Darum: oft committen.

Häufige Fehler mit Git

- **fatal: not a git repository** . Du bist nicht im Repo. Lösung: `cd ~/linux-kurs` .
- **Author identity unknown** . Name und E-Mail fehlen. Lösung: die zwei `git config` -Zeilen von oben.
- **nothing added to commit oder no changes added to commit** . Du hast `git add` vergessen.

- **Es öffnet sich nano.** Du hast `-m` vergessen. Schreibe die Nachricht in die erste Zeile. Speichere mit `Strg+O` und `Enter`. Beende mit `Strg+X`.
- **Unten steht : oder (END)** . Git zeigt eine lange Ausgabe mit `less` . Drücke `q` .

Aufgaben · Teil 1

Git einrichten

Wozu? Git schreibt in jeden Commit, wer ihn gemacht hat. Das stellst du einmal ein. Danach nie wieder.

1. Aktualisiere die Paketlisten. Tippe im Terminal: `sudo apt update` . `sudo` fragt nach deinem Linux-Passwort. Beim Tippen siehst du nichts.
2. Installiere Git: `sudo apt install git` . Fragt apt `Do you want to continue? [Y/n]` , drücke `Enter`.
3. Prüfe die Version: `git --version`
4. Trage deinen Namen ein: `git config --global user.name "Lucia Garcia"` . Schreibe deinen Namen statt Lucia Garcia.
5. Trage deine E-Mail ein: `git config --global user.email "lucia@example.com"` . Nimm die E-Mail, die du für GitHub nutzt.
6. Gib der Reihe deiner Commits den Namen `main` : `git config --global init.defaultBranch main`
7. Prüfe alles: `git config --list`

So sieht das Ende aus:

```
user.name=Lucia Garcia
user.email=lucia@example.com
init.defaultbranch=main
```

Git schreibt `defaultbranch` klein. Das ist normal.

✓ **Bestätigt**

Dein Kursordner wird ein Repo

Wozu? Deine Arbeit aus Woche 1 liegt in `~/linux-kurs`. Ab heute speichert Git jede Version davon. Nach der großen Pause lädst du alles auf GitHub.

1. Lege den Ordner an: `mkdir -p ~/linux-kurs`. Gibt es ihn schon, passiert nichts.
2. Wechsle hinein: `cd ~/linux-kurs`
3. Mach daraus ein Repo: `git init`
4. Zeige alle Dateien, auch die versteckten: `ls -a`. Jetzt gibt es den Ordner `.git`.
5. Frage Git, was es sieht: `git status`. Deine Ordner stehen unter `Untracked files`.
6. Wähle alles aus: `git add .`
7. Frage noch einmal: `git status`. Jetzt stehen deine Dateien unter `Changes to be committed`.
8. Speichere die erste Version: `git commit -m "Add work from week 1"`
9. Zeige den Verlauf: `git log --oneline`

So sieht das Ende aus. Die Nummer vorne ist bei dir anders:

```
d1fde11 (HEAD -> main) Add work from week 1
```

Ist dein Ordner leer, meldet `git commit` den Fehler `nothing to commit`. Dann lege eine Datei an: `echo "Linux-Kurs" > notiz.txt`. Mach danach ab Schritt 6 weiter.

✓ **Bestätigt**

Ändern, vergleichen, zurückholen

Wozu? Beim Arbeiten machst du Fehler. Git zeigt dir, was du geändert hast. Und Git holt eine kaputte Datei zurück.

1. Lege den Ordner für heute an: `mkdir -p ~/linux-kurs/tag06`
2. Wechsle hinein: `cd ~/linux-kurs/tag06`
3. Öffne eine neue Datei: `nano hallo.sh`
4. Tippe diese 3 Zeilen:

```
#!/bin/bash
# Gruesst den Benutzer
echo "Hallo $USER"
```

5. Speichere mit Strg+O und Enter. Beende mit Strg+X.
6. Mache das Skript ausführbar: `chmod +x hallo.sh`
7. Wähle die Datei aus: `git add hallo.sh`
8. Speichere die Version: `git commit -m "Add hello script"`
9. Öffne die Datei noch einmal: `nano hallo.sh`
10. Tippe in eine neue letzte Zeile: `echo "Bis morgen"` . Speichere und beende.
11. Zeige die Änderung: `git diff` . Unten steht `+echo "Bis morgen"` .
12. Wähle die Datei aus: `git add hallo.sh`
13. Speichere die zweite Version: `git commit -m "Add goodbye message"`
14. Jetzt machst du absichtlich einen Fehler. Tippe mit nur einem `>` : `echo "kaputt" >`
`hallo.sh`
15. Schau in die Datei: `cat hallo.sh` . Darin steht nur noch `kaputt` .
16. Hole die Datei zurück: `git restore hallo.sh`
17. Schau noch einmal: `cat hallo.sh` . Alle 4 Zeilen sind wieder da.
18. Zeige den Verlauf: `git log --oneline`

So sieht das Ende aus:

```
09848b4 (HEAD -> main) Add goodbye message
adfc1ed Add hello script
d1fde11 Add work from week 1
```

✓ Bestätigt

Quiz · Teil 1

Das Quiz gibt keine Punkte. Du kannst es so oft wiederholen, wie du willst.

Quiz abgegeben. Ergebnis: 4 / 4 richtig

1. Du hast `hallo.sh` geändert. Wie speicherst du die Änderung als Version?

erst `git add hallo.sh`, dann `git commit -m "Add hello script"` (Deine Antwort)

erst `git commit -m "Add hello script"`, dann `git add hallo.sh`

erst `git add hallo.sh`, dann `git push`

nur `git init`

Richtig

Erklärung:

`git add` wählt die Änderung aus. `git commit` speichert sie als Version. `git push` lädt nur hoch.

2. Welcher Befehl macht aus dem aktuellen Ordner ein Git-Repo?

Deine Antwort: `git init`

Richtig

Erklärung:

`git init` legt den versteckten Ordner `.git` an. Darin liegen alle Versionen.

3. Welche Commit-Nachricht ist richtig geschrieben?

Added backup script

Add backup script (Deine Antwort)

changes

backup.sh

Richtig

Erklärung:

Die Nachricht steht im Imperativ: Add, nicht Added. Und sie sagt, was der Commit macht.

4. Du hast `hallo.sh` aus Versehen mit `>` überschrieben. Im letzten Commit ist die gute Version. Welcher Befehl holt sie zurück?

`git diff hallo.sh`

`git add hallo.sh`

`git restore hallo.sh` (Deine Antwort)

`git log hallo.sh`

Richtig

Erklärung:

`git restore` holt die Datei so zurück, wie sie im letzten Commit war. `git diff` zeigt die Änderung nur an.

Teil 2: Alte Stände und Branches nach der ersten Pause

Wo bist du gerade?

In `git log --oneline` steht beim neuesten Commit (`HEAD -> main`) . **HEAD** zeigt, wo du gerade bist. `HEAD -> main` heißt: Du bist auf dem Branch `main` , beim neuesten Commit. Die Dateien in deinem Ordner haben diesen Stand.

Mit `git switch` bewegst du HEAD. Dann ändern sich die Dateien in deinem Ordner. `git switch` gibt es seit 2019. In älteren Anleitungen steht dafür `git checkout` . Das macht dasselbe. `git switch` ist aber klarer.

Einen alten Stand ansehen

Dein Skript läuft heute anders als gestern. Was war gestern anders? Dafür springst du zu einem alten Commit. Dort startest du das alte Skript und schaust dir die Dateien an. Danach springst du zurück.

Die Nummer des alten Commits steht in `git log --oneline` . Mit ihr springst du zurück:

```
git switch --detach adfc1ed
# HEAD is now at adfc1ed Add hello script
./hallo.sh
# Hallo carlos
```

Alle Dateien im Ordner haben jetzt den alten Stand. Die Zeile `Bis morgen` fehlt. Sie ist nicht verloren. Sie steckt im neueren Commit.

Was ist dabei passiert? Das zeigt `git log --oneline --all` . Vorher:

```
git log --oneline --all
# 09848b4 (HEAD -> main) Add goodbye message
# adfc1ed Add hello script
# d1fde11 Add work from week 1
```

Nachher:

```
git log --oneline --all
# 09848b4 (main) Add goodbye message
# adfc1ed (HEAD) Add hello script
# d1fde11 Add work from week 1
```

`main` bleibt beim neuesten Commit. `HEAD` steht jetzt allein beim alten Commit, ohne Branch. Das heißt **detached HEAD**, auf Deutsch: losgelöst. `git status` zeigt dann `HEAD detached at adfc1ed`.

Warum brauchst du `--detach`? `git switch` wechselt normal zu einem Branch. `adfc1ed` ist aber kein Branch, sondern ein Commit. Ohne `--detach` sagt Git darum nein. Mit `--detach` sagst du: Ich weiß, ich verlasse den Branch. Ich will nur schauen.

Mach hier keinen Commit. Er gehört zu keinem Branch. Nach dem Zurückspringen findest du ihn nicht mehr. Zurück zum neuesten Stand geht es so:

```
git switch main
# Previous HEAD position was adfc1ed Add hello script
# Switched to branch 'main'
```

Eine alte Datei zurückholen

Du willst die alte `hallo.sh` wirklich zurück. Dann bleibst du auf `main` und holst nur diese Datei aus dem alten Commit:

```
git restore --source=adfc1ed hallo.sh
git add hallo.sh
git commit -m "Remove goodbye message"
```

`--source` sagt Git, aus welchem Commit die Datei kommt. Danach machst du einen neuen Commit. Der Verlauf bleibt ganz. Auch die Zeile `echo "Bis morgen"` ist darin noch gespeichert.

Branches

Bis jetzt liegen alle Commits in einer Reihe. Sie heißt `main`. Du willst etwas Neues ausprobieren. Es soll `main` nicht stören. Dafür legst du einen **Branch** an. Ein neuer Branch ist eine zweite Reihe von Commits. Sie beginnt beim Stand, auf dem du gerade bist.

```
git switch -c wochentag
# Switched to a new branch 'wochentag'
```

`-c` heißt create, also neu anlegen. Neue Commits landen jetzt nur im Branch `wochentag`. `git branch` zeigt alle Branches. Der Stern steht bei dem, auf dem du bist:

```
git branch
#   main
# * wochentag
```

Wechselst du mit `git switch main` zurück, fehlen in den Dateien die Änderungen aus `wochentag`. `main` hat noch den alten Stand. So arbeiten auch Firmen: Jeder probiert in seinem Branch. `main` bleibt immer lauffähig.

Einen Branch zusammenführen

Der Versuch hat geklappt. Jetzt sollen die Commits auch in `main`. Das heißt **merge**, auf Deutsch zusammenführen. Du gehst dafür auf `main` und holst den Branch dazu:

```
git switch main
git merge wochentag
# Updating 7b2e4d1..3c1d2e7
# Fast-forward
```

Fast-forward heißt: `main` hatte keine eigenen neuen Commits. Git hängt die Commits aus `wochentag` einfach an.

Danach brauchst du den Branch nicht mehr. Du löschst ihn mit `git branch -d wochentag`. Die Commits bleiben in `main`.

Hat der Versuch nicht geklappt? Dann wechselst du zu `main` und löschst den Branch mit `git branch -D wochentag`. Das große `-D` löscht auch Commits, die nicht in `main` sind.

Häufige Fehler mit Branches

- **Your local changes ... would be overwritten**. Du hast etwas geändert, aber nicht committet. Dann wechselt Git nicht. Oder Git wechselt und nimmt die Änderung mit. Darum: vor `git switch` immer erst `git add` und `git commit`.
- **a branch is expected, got commit**. Du hast `--detach` vergessen. Richtig: `git switch --detach adfc1ed`.
- **invalid reference oder could not resolve**. Tippfehler im Namen oder in der Nummer. Die Nummern zeigt `git log --oneline`, die Namen zeigt `git branch`.

- **git status zeigt HEAD detached at .** Du bist noch auf einem alten Stand. Zurück mit `git switch main` .
- **not fully merged .** Der Branch hat Commits, die `main` nicht hat. Erst `git merge` , dann `git branch -d` .

Aufgaben · Teil 2

Zurück zu einem alten Stand

Wozu? Manchmal willst du wissen, wie dein Projekt früher war. Lief das Skript damals anders? Git hat jeden Stand gespeichert. Du springst hin, schaust nach und springst zurück. Eine alte Datei kannst du auch ganz zurückholen.

1. Wechsle in den Ordner von heute: `cd ~/linux-kurs/tag06`
2. Zeige den Verlauf: `git log --oneline`. Suche die Zeile mit `Add hello script`. Die Nummer vorne brauchst du gleich. In den nächsten Schritten steht dafür `adfc1ed`. Tippe immer deine Nummer.
3. Starte das Skript: `./hallo.sh`. Es gibt zwei Zeilen aus.
4. Springe zum alten Stand: `git switch --detach adfc1ed`
5. Zeige, wo du jetzt bist: `git log --oneline --all`. (HEAD) steht beim alten Commit. (main) steht beim neuesten.
6. Starte das alte Skript: `./hallo.sh`. Jetzt kommt nur eine Zeile. Bis morgen fehlt.
7. Frage Git: `git status`. Oben steht `HEAD detached at` und deine Nummer.
8. Springe zurück zum neuesten Stand: `git switch main`
9. Starte das Skript noch einmal: `./hallo.sh`. Beide Zeilen sind wieder da.
10. Hole jetzt die alte Datei wirklich zurück: `git restore --source=adfc1ed hallo.sh`. Tippe wieder deine Nummer.
11. Schau in die Datei: `cat hallo.sh`. Die Zeile `echo "Bis morgen"` fehlt jetzt.
12. Wähle die Datei aus: `git add hallo.sh`
13. Speichere die Version: `git commit -m "Remove goodbye message"`
14. Zeige den Verlauf: `git log --oneline`

So sieht Schritt 5 aus. Die Nummern sind bei dir anders:

```
09848b4 (main) Add goodbye message
adfc1ed (HEAD) Add hello script
d1fde11 Add work from week 1
```

So beginnt der Verlauf am Ende:

```
7b2e4d1 (HEAD -> main) Remove goodbye message
09848b4 Add goodbye message
adfc1ed Add hello script
```

✓ Bestätigt

Ein Branch zum Ausprobieren

Wozu? Du willst etwas Neues ausprobieren. Dein Skript auf `main` soll dabei heil bleiben. In einem Branch probierst du ohne Risiko. Klappt es, holst du es nach `main`.

1. Wechsle in den Ordner von heute: `cd ~/linux-kurs/tag06`
2. Lege den Branch `wochentag` an und wechsle hinein: `git switch -c wochentag`
3. Zeige alle Branches: `git branch`. Der Stern steht bei `wochentag`.
4. Öffne die Datei: `nano hallo.sh`
5. Tippe in eine neue letzte Zeile: `echo "Heute ist $(date +%A)"`. Speichere und beende.
6. Starte das Skript: `./hallo.sh`
7. Wähle die Datei aus: `git add hallo.sh`
8. Speichere die Version: `git commit -m "Add weekday"`
9. Frage Git: `git status`. Dort steht `nothing to commit, working tree clean`.
10. Wechsle zurück: `git switch main`
11. Schau in die Datei: `cat hallo.sh`. Die neue Zeile fehlt. Sie ist nur im Branch `wochentag`.
12. Hole den Branch nach `main`: `git merge wochentag`
13. Schau noch einmal: `cat hallo.sh`. Die neue Zeile ist da.
14. Lösche den Branch: `git branch -d wochentag`
15. Zeige den Verlauf: `git log --oneline`

So sieht der Start in Schritt 6 aus. Statt `Monday` steht bei dir der heutige Tag:

```
Hallo carlos
Heute ist Monday
```

So beginnt der Verlauf am Ende. Die Nummern sind bei dir anders:

```
3c1d2e7 (HEAD -> main) Add weekday
7b2e4d1 Remove goodbye message
09848b4 Add goodbye message
```

✓ Bestätigt

Quiz · Teil 2

Das Quiz gibt keine Punkte. Du kannst es so oft wiederholen, wie du willst.

Quiz abgegeben. Ergebnis: 4 / 4 richtig

1. Was heißt (HEAD -> main) in `git log --oneline` ?

Der Commit ist schon auf GitHub.

Du bist gerade auf dem Branch `main`, bei diesem Commit. (Deine Antwort)

`main` ist der älteste Commit.

HEAD ist der Name deines Repos.

Richtig

Erklärung:

HEAD zeigt, wo du gerade bist. Die Dateien in deinem Ordner haben den Stand dieses Commits.

2. Mit welchem Befehl legst du den Branch `test` an und wechselst gleich hinein?

Deine Antwort: `git switch -c test`

Richtig

Erklärung:

`-c` heißt create. `git switch -c` legt den Branch an und wechselt hinein.

3. Du bist auf `main` . Wie holst du die Commits aus dem Branch `wochentag` nach `main` ?

```
git switch wochentag
```

```
git branch wochentag
```

```
git restore wochentag
```

```
git merge wochentag (Deine Antwort)
```

Richtig

Erklärung:

`git merge` führt den Branch mit dem Branch zusammen, auf dem du gerade bist. `git switch` wechselt nur.

4. Du willst nur ansehen, wie dein Ordner beim Commit `adfc1ed` aussah. Welcher Befehl passt?

```
git switch --detach adfc1ed (Deine Antwort)
```

```
git merge adfc1ed
```

```
git branch -d adfc1ed
```

```
git commit adfc1ed
```

Richtig

Erklärung:

`--detach` springt zu einem alten Commit, ohne Branch. Zurück geht es mit `git switch main` .

Teil 3: Dein Repo auf GitHub nach der großen Pause

Was ist GitHub?

GitHub ist eine Webseite. Dort liegt eine Kopie deines Repos, mit allen Commits. Das bringt dir zwei Dinge:

- Geht dein Rechner kaputt, ist deine Arbeit noch da.
- Andere können dein privates Repo sehen, wenn du sie einlädst. Das geht im Repo unter **Settings**, **Collaborators**.

Git ist das Programm auf deinem Rechner. GitHub ist ein Dienst im Internet, der Git-Repos speichert. Es gibt auch andere, zum Beispiel GitLab und Codeberg.

GitHub startete 2008. Seit 2018 gehört es Microsoft. Microsoft zahlte dafür 7,5 Milliarden Dollar. Heute sind über 180 Millionen Entwickler auf GitHub.

Konto und leeres Repo

Öffne GitHub im Firefox auf deinem Linux. Sonst kannst du deinen Schlüssel später nicht vom Terminal zu GitHub kopieren. Das Konto legst du auf <https://github.com> an, mit **Sign up**. Du brauchst eine E-Mail-Adresse, ein Passwort und einen Benutzernamen. GitHub schickt dir einen Code per E-Mail. Den gibst du ein. Meldest du dich später in einem neuen Browser an, kommt wieder ein Code per E-Mail.

Ein neues Repo legst du oben rechts mit **+** und **New repository** an. Es heißt `linux-kurs`, ist **Private** und bleibt leer, also ohne README. Danach zeigt GitHub unter **Quick setup** seine Adresse. Wähle dort **SSH**. Die Adresse sieht so aus: `git@github.com:carlos-ruiz/linux-kurs.git`.

Anmelden mit SSH

Beim Hochladen muss GitHub wissen: Bist das wirklich du? Dein GitHub-Passwort geht dafür nicht. GitHub hat das im August 2021 abgeschaltet. Wir nehmen **SSH**.

SSH arbeitet mit zwei Schlüsseln. Das sind zwei Dateien, die zusammengehören:

- Der **private Schlüssel** bleibt auf deinem Rechner. Den gibst du nie weiter.
- Den **öffentlichen Schlüssel** gibst du GitHub. Seine Datei endet auf `.pub`.

Beim Push prüft GitHub, ob dein privater Schlüssel zum öffentlichen passt. Dann weiß GitHub: Das bist du. Ein Passwort brauchst du nicht.

SSH gibt es seit 1995. Tatu Ylönen hat es in Finnland entwickelt. Admins nutzen es jeden Tag, um sich auf Servern anzumelden.

Die Schlüssel machst du einmal pro Rechner:

```
ssh-keygen -t ed25519 -C "carlos@example.com"
```

-t ed25519 ist die Art des Schlüssels. GitHub empfiehlt sie. -C schreibt deine E-Mail als Notiz dazu. Danach fragt ssh-keygen dreimal. Drücke jedes Mal nur Enter. Zweimal fragt es nach einem Passwort für den Schlüssel. Tippe keins, auch nicht dein GitHub-Passwort. Fragt es `overwrite (y/n)?`, hast du schon einen Schlüssel. Dann tippe `n`. Jetzt liegen zwei neue Dateien in `~/.ssh`: `id_ed25519` und `id_ed25519.pub`.

Den öffentlichen Schlüssel zeigt `cat ~/.ssh/id_ed25519.pub`. Die Tab-Taste ergänzt nur bis `id_ed25519`. Tippe `.pub` selbst dazu. Diese eine Zeile kopierst du zu GitHub: Oben rechts auf dein Bild, dann **Settings**, links **SSH and GPG keys**, dann **New SSH key**. Oder direkt über <https://github.com/settings/keys>.

Achtung: Kopiere nur die Datei mit `.pub`. Die Datei ohne `.pub` ist dein privater Schlüssel. Wer sie hat, kann sich als du anmelden.

Prüfen:

```
ssh -T git@github.com
# Hi carlos-ruiz! You've successfully authenticated, but GitHub does not provide shell acce
```

Beim ersten Mal fragt SSH: `Are you sure you want to continue connecting (yes/no/[fingerprint])?`. Tippe `yes` und Enter. SSH merkt sich GitHub dann in der Datei `~/.ssh/known_hosts` und fragt nicht mehr.

Hochladen mit push

```
git remote add origin git@github.com:carlos-ruiz/linux-kurs.git
git push -u origin main
```

- `git remote add` merkt sich die Adresse auf GitHub unter dem Namen `origin`. Nimm die Adresse mit **SSH**. Sie beginnt mit `git@github.com:`.
- `git push` lädt deine Commits hoch. Das `-u` merkt sich das Ziel. Danach reicht `git push`.

Git fragt nicht nach einem Passwort. Der Schlüssel reicht.

`git push` lädt hoch. `git pull` holt herunter, was auf GitHub neu ist.

Ein Repo klonen

Du hast einen neuen Rechner. Oder deine VM ist kaputt. Dein Repo liegt aber noch auf GitHub. Mit `git clone` holst du es zurück, mit allen Commits:

```
cd /tmp
git clone git@github.com:carlos-ruiz/linux-kurs.git
# Cloning into 'linux-kurs'...
cd linux-kurs
git log --oneline
```

`git clone` legt einen neuen Ordner an. Er heißt wie das Repo, hier `linux-kurs`. Darin liegen alle Dateien und der ganze Verlauf. Die Adresse steht schon als `origin` drin. Du kannst sofort `git pull` und `git push` benutzen.

Die Adresse findest du auf GitHub beim grünen Knopf **Code**. Wähle dort **SSH**. Auf einem neuen Rechner brauchst du vorher einen neuen SSH-Schlüssel.

Klonen machst du einmal pro Rechner. Danach holst du neue Commits mit `git pull`.

Was nicht auf GitHub gehört

Passwörter und private Schlüssel gehören nie in ein Repo, auch nicht in ein `private`. Was einmal in einem Commit steht, bleibt im Verlauf.

Dateien, die Git übersehen soll, schreibst du in die Datei `.gitignore` im Repo. Eine Zeile pro Muster, zum Beispiel `*.log` für alle Logdateien. `git add .` nimmt diese Dateien dann nicht mit.

Häufige Fehler mit GitHub

- **Permission denied (publickey)** . GitHub kennt deinen Schlüssel nicht. Prüfe unter **Settings, SSH and GPG keys**, ob er dort steht. Hast du die Zeile aus der Datei mit `.pub` kopiert?
- **Username for 'https://github.com'** . Du hast die Adresse mit `https://` genommen. Drücke Strg+C. Stelle auf SSH um: `git remote set-url origin git@github.com:carlos-ruiz/linux-kurs.git` , mit deinem GitHub-Namen.
- **Key is invalid** . Du hast die falsche Datei oder nicht die ganze Zeile kopiert. Die Zeile beginnt mit `ssh-ed25519` und endet mit deiner E-Mail.
- **Repository not found** . Die Adresse hat einen Fehler. Prüfe sie mit `git remote -v` und ändere sie mit `git remote set-url origin` .
- **destination path 'linux-kurs' already exists** . Hier gibt es schon einen Ordner `linux-kurs` . Wechsle vor dem Klonen in einen anderen Ordner, zum Beispiel mit `cd /tmp` .
- **remote origin already exists** . Du hast `git remote add` zweimal getippt. Prüfe die Adresse mit `git remote -v` .
- **[rejected] main -> main (fetch first)** . Auf GitHub liegt etwas, das du nicht hast. Meist hat das Repo eine README. Lösung: `git pull --rebase origin main` , dann noch einmal `git push -u origin main` .
- **ssh -T wartet lange oder meldet Connection timed out** . Drücke nach 30 Sekunden Strg+C. Das Netz sperrt SSH auf Port 22. GitHub erlaubt SSH auch über Port 443. Tippe `nano ~/.ssh/config` und schreibe diese vier Zeilen:

```
Host github.com
  Hostname ssh.github.com
  Port 443
  User git
```

Speichere mit Strg+O und Enter. Beende mit Strg+X. Tippe noch einmal `ssh -T git@github.com` . SSH fragt noch einmal. Tippe `yes` .

Aufgaben • Teil 3

Konto, Repo und SSH-Schlüssel

Wozu? GitHub hebt eine Kopie deiner Arbeit auf. Dafür brauchst du ein Konto und ein leeres Repo. Mit einem SSH-Schlüssel erkennt GitHub dich ohne Passwort.

1. Öffne `https://github.com` im Firefox auf deinem Linux. Sonst kannst du den Schlüssel nicht vom Terminal kopieren. Hast du schon ein Konto? Dann melde dich an und mach bei Schritt 4 weiter.
2. Klicke auf **Sign up**. Gib E-Mail, Passwort und Benutzernamen ein.
3. GitHub schickt dir einen Code per E-Mail. Gib ihn ein.
4. Klicke oben rechts auf **+**, dann auf **New repository**.
5. Bei **Repository name** tippst du `linux-kurs`.
6. Bei **Choose visibility** wählst du **Private**.
7. **Add README** bleibt auf **Off**. Bei `.gitignore` und Lizenz wählst du nichts.
8. Klicke auf **Create repository**. Lass diese Seite offen.
9. Mache im Terminal deine Schlüssel. Schreibe deine GitHub-E-Mail statt `carlos@example.com`:

```
ssh-keygen -t ed25519 -C "carlos@example.com"
```
10. `ssh-keygen` fragt dreimal. Drücke jedes Mal nur Enter. Zweimal fragt es nach einem Passwort für den Schlüssel. Tippe keins. Fragt es `overwrite (y/n)?`, hast du schon einen Schlüssel. Dann tippe `n` und Enter.
11. Zeige den öffentlichen Schlüssel: `cat ~/.ssh/id_ed25519.pub`. Tippe `.pub` ganz aus. Die Tab-Taste hört vorher auf.
12. Markiere die ganze Zeile mit der Maus. Sie geht über zwei Zeilen. Sie beginnt mit `ssh-ed25519` und endet mit deiner E-Mail. Kopiere sie mit `Strg+Umschalt+C`.
13. Öffne im Firefox einen neuen Tab mit `Strg+T`. Gehe auf `https://github.com/settings/keys`
14. Klicke auf **New SSH key**.
15. Bei **Title** tippst du `Kurs-VM`. **Key type** bleibt auf **Authentication Key**.
16. Klicke in das Feld **Key**. Füge die Zeile mit `Strg+V` ein.
17. Klicke auf **Add SSH key**. Zeigt GitHub **Confirm access**, gib dein GitHub-Passwort ein.
18. Prüfe im Terminal: `ssh -T git@github.com`
19. Fragt SSH `Are you sure you want to continue connecting`, tippe `yes` und Enter.
20. Kommt `Permission denied (publickey)` oder wartet SSH lange? Dann lies im Text **Häufige Fehler mit GitHub**.

So sieht das Ende aus. Statt `carlos-ruiz` steht dort dein GitHub-Name:

Hi carlos-ruiz! You've successfully authenticated, but GitHub does not provide shell access.

✓ Bestätigt

Hochladen und jemanden einladen

Wozu? Jetzt kommt deine Arbeit auf GitHub. Wer eingeladen ist, kann sie dort sehen.

1. Wechsle ins Repo: `cd ~/linux-kurs`
2. Gehe im Browser auf die Seite deines Repos. Klicke unter **Quick setup** auf **SSH**. Klicke auf das Kopier-Symbol rechts neben der Adresse.
3. Tippe im Terminal `git remote add origin`, dann ein Leerzeichen. Füge die Adresse mit Strg+Umschalt+V ein. Drücke Enter.
4. Prüfe die Adresse: `git remote -v`. Sie beginnt mit `git@github.com:`. Steht dort `https://`, lies im Text **Häufige Fehler mit GitHub**.
5. Lade alles hoch: `git push -u origin main`. Git fragt nicht nach einem Passwort.
6. Lade die Seite im Browser neu. Deine Ordner sind da. Leere Ordner fehlen. Git speichert nur Dateien.
7. Klicke im Repo oben auf **Settings**. Klicke links auf **Collaborators**.
8. Klicke auf **Add people**. Tippe `MiguelitoMe` und wähle den Namen aus der Liste.
9. Klicke auf **Add MiguelitoMe to linux-kurs**.

So endet der Push:

```
To github.com:carlos-ruiz/linux-kurs.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Die Einladung muss noch angenommen werden. Danach sieht `MiguelitoMe` dein Repo.

✓ Bestätigt

Holen und wieder hochladen

Wozu? Manchmal änderst du etwas auf der Webseite oder an einem anderen Rechner. Mit `git pull` holst du das zu dir. Mit `git push` lädst du deine Arbeit hoch.

1. Öffne dein Repo `linux-kurs` im Browser.
2. Klicke auf **Add file**, dann auf **Create new file**.
3. Tippe oben als Namen: `README.md`
4. Tippe in das große Feld: `# Linux-Kurs`
5. Tippe in die nächste Zeile: `Meine Aufgaben aus dem Linux-Kurs`. Drücke am Ende `Enter`.
6. Klicke auf **Commit changes...**. Im Fenster klickst du noch einmal auf **Commit changes**.
7. Wechsle im Terminal ins Repo: `cd ~/linux-kurs`
8. Hole die Änderung: `git pull`
9. Prüfe die Datei: `cat README.md`
10. Öffne die Datei: `nano README.md`
11. Tippe in eine neue letzte Zeile: `Seit Tag 6 mit Git`. Speichere und beende.
12. Wähle alles aus: `git add .`
13. Speichere die Version: `git commit -m "Update readme"`
14. Lade hoch: `git push`
15. Lade die Seite im Browser neu. Deine neue Zeile ist da.

So beginnt `git log --oneline` am Ende. Die Nummern sind bei dir anders:

```
5c3e8a0 (HEAD -> main, origin/main) Update readme
b41f7d2 Create README.md
3c1d2e7 Add weekday
```

✓ **Bestätigt**

Dein Repo klonen

Wozu? Auf einem neuen Rechner holst du dein ganzes Repo mit einem Befehl. Das brauchst du, wenn deine VM kaputtgeht. Heute spielt der Ordner `/tmp` den zweiten Rechner. Linux leert `/tmp` beim Neustart.

1. Gehe im Browser auf dein Repo. Klicke auf den grünen Knopf **Code**. Klicke im Fenster auf **SSH**, dann auf das Kopier-Symbol.
2. Wechsle nach `/tmp`: `cd /tmp`
3. Tippe `git clone`, dann ein Leerzeichen. Füge die Adresse mit Strg+Umschalt+V ein. Drücke Enter.
4. Wechsle in die Kopie: `cd /tmp/linux-kurs`
5. Zeige den Verlauf: `git log --oneline`. Es sind dieselben Commits wie in `~/linux-kurs`.
6. Zeige die Adresse: `git remote -v`. `origin` ist schon eingetragen.
7. Hänge eine Zeile an: `echo "Auch in /tmp geklont." >> README.md`
8. Wähle die Datei aus: `git add README.md`
9. Speichere die Version: `git commit -m "Add note from second copy"`
10. Lade hoch: `git push`
11. Wechsle zurück in dein Repo: `cd ~/linux-kurs`
12. Hole die Änderung: `git pull`
13. Prüfe: `cat README.md`. Die neue Zeile ist da.

Git fragt beim Push nicht nach einem Passwort. Dein SSH-Schlüssel gilt für alle deine Repos.

✓ Bestätigt

Quiz · Teil 3

Das Quiz gibt keine Punkte. Du kannst es so oft wiederholen, wie du willst.

Quiz abgegeben. Ergebnis: **5 / 5 richtig**

1. Was ist der Unterschied zwischen Git und GitHub?

Git ist eine Webseite, GitHub ist ein Programm auf deinem Rechner.

Git ist die alte Version von GitHub.

Es ist dasselbe, nur mit zwei Namen.

Git ist ein Programm auf deinem Rechner, GitHub ist ein Dienst im Internet. (Deine Antwort)

Richtig

Erklärung:

Git speichert die Versionen auf deinem Rechner. GitHub hebt eine Kopie deines Repos im Internet auf.

2. Welche Datei gibst du GitHub, damit es dich per SSH erkennt?

~/.ssh/id_ed25519

~/.ssh/id_ed25519.pub (Deine Antwort)

~/.ssh/known_hosts

~/.bashrc

Richtig

Erklärung:

Die Datei mit `.pub` ist der öffentliche Schlüssel. Die Datei ohne `.pub` ist privat und bleibt auf deinem Rechner.

3. Mit welchem Befehl lädst du deine Commits das erste Mal nach origin auf main hoch?

Deine Antwort: `git push -u origin main`

Richtig

Erklärung:

Das `-u` merkt sich das Ziel. Danach reicht `git push`.

4. Du hast auf der Webseite von GitHub eine Datei geändert. Wie holst du die Änderung in deinen Ordner ~/linux-kurs ?

`git push`

`git commit`

`git pull` (Deine Antwort)

`git init`

Richtig

Erklärung:

`git pull` holt herunter, was auf GitHub neu ist. `git push` lädt in die andere Richtung hoch.

5. Deine VM ist kaputt. Auf dem neuen Rechner willst du dein Repo von GitHub holen. Welcher Befehl passt?

```
git pull
```

```
git init
```

```
git clone git@github.com:carlos-ruiz/linux-kurs.git (Deine Antwort)
```

```
git remote add origin git@github.com:carlos-ruiz/linux-kurs.git
```

Richtig

Erklärung:

`git clone` legt einen neuen Ordner an und holt alle Commits. `git pull` geht nur in einem Repo, das es auf dem Rechner schon gibt.

Bonusaufgaben

Freiwillig, für alle, die früher fertig sind. Bonusaufgaben zählen nicht für die Tagesmedaille. Sind alle Bonusaufgaben eines Tages bestätigt, gibt es eine Bonusmedaille.

Logdateien übersehen Bonus

Wozu? Manche Dateien sollen nie auf GitHub. Zum Beispiel Logdateien oder Passwörter. Die Datei `.gitignore` hält sie draußen.

1. Wechsle ins Repo: `cd ~/linux-kurs`
2. Lege eine Logdatei an: `echo "Test" > tag06/test.log`
3. Frage Git: `git status`. Unter Untracked files steht `tag06/test.log`.
4. Öffne eine neue Datei: `nano .gitignore`
5. Tippe in die erste Zeile: `*.log`
6. Speichere mit Strg+O und Enter. Beende mit Strg+X.
7. Frage Git noch einmal: `git status`. `test.log` ist weg. Dafür steht dort `.gitignore`.
8. Wähle die Datei aus: `git add .gitignore`
9. Speichere die Version: `git commit -m "Ignore log files"`
10. Lade hoch: `git push`

So sieht `git status` nach Schritt 7 aus:

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
```

✓ Bestätigt

Ein Konflikt beim Merge Bonus

Wozu? Manchmal ändern zwei Branches dieselbe Zeile. Dann weiß Git nicht, welche Zeile gilt. Das heißt **Konflikt**. Du entscheidest dann selbst.

1. Wechsle in den Ordner von heute: `cd ~/linux-kurs/tag06`
2. Lege den Branch `gruss` an und wechsle hinein: `git switch -c gruss`
3. Öffne die Datei: `nano hallo.sh`. Ändere in der Zeile `echo "Hallo $USER"` das Wort `Hallo` in `Hi`. Speichere und beende.
4. Wähle die Datei aus: `git add hallo.sh`
5. Speichere die Version: `git commit -m "Say hi"`
6. Wechsle zurück: `git switch main`
7. Öffne die Datei: `nano hallo.sh`. Ändere in derselben Zeile das Wort `Hallo` in `Servus`. Speichere und beende.
8. Wähle die Datei aus: `git add hallo.sh`
9. Speichere die Version: `git commit -m "Say servus"`
10. Führe zusammen: `git merge gruss`. Git meldet `CONFLICT`.
11. Schau in die Datei: `cat hallo.sh`. Git hat beide Zeilen hineingeschrieben, mit Markierungen.
12. Öffne die Datei: `nano hallo.sh`
13. Lösche die drei Zeilen `<<<<<< HEAD`, `=====` und `>>>>>> gruss`.
14. Lösche eine der zwei echo-Zeilen mit dem Gruß. Behalte die, die dir besser gefällt.
15. Speichere mit Strg+O und Enter. Beende mit Strg+X.
16. Wähle die Datei aus: `git add hallo.sh`
17. Speichere den Merge: `git commit -m "Merge branch gruss"`
18. Lösche den Branch: `git branch -d gruss`
19. Zeige den Verlauf als Bild: `git log --oneline --graph`
20. Lade hoch: `git push`

So sieht `hallo.sh` nach Schritt 10 aus:

```
#!/bin/bash
# Gruesst den Benutzer
<<<<<< HEAD
echo "Servus $USER"
=====
echo "Hi $USER"
```

```
>>>>>> gruss  
echo "Heute ist $(date +%A)"
```

✓ Bestätigt